

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнила:

Студент 4 курса

Группы ИИ-26

Карпович К.А.

Проверила:

Андренко К.В.

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Общее задание:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать torchvision.datasets). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
6	MNIST	28X28	Adam

Для решения задачи классификации рукописных цифр из набора данных MNIST (28x28, 1 канал) была спроектирована сверточная нейронная сеть со следующей архитектурой:

1. Сверточный блок 1: Conv2d (1→16 каналов, ядро 3 x 3, padding=1) → ReLU → MaxPool2d (2 x 2).
2. Сверточный блок 2: Conv2d (16 → 32 каналов, ядро 3 x 3, padding=1) → ReLU → MaxPool2d (2 x 2).
3. Полносвязный блок: Выравнивание вектора (32 x 7 x 7 = 1568) → Linear (1568 → 128) → ReLU → Linear (128 → 10).

Код программы:

```
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
import numpy as np

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используем устройство: {device}")

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=1000, shuffle=False)

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
```

```

# Сверточный блок 1: [1, 28, 28] -> [16, 28, 28] -> [16, 14, 14]
self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
self.relu = nn.ReLU()
self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

# Сверточный блок 2: [16, 14, 14] -> [32, 14, 14] -> [32, 7, 7]
self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)

# Полносвязные слои: [32 * 7 * 7] -> 128 -> 10
self.fc1 = nn.Linear(32 * 7 * 7, 128)
self.fc2 = nn.Linear(128, 10)

def forward(self, x):
    x = self.pool(self.relu(self.conv1(x)))
    x = self.pool(self.relu(self.conv2(x)))
    x = x.view(x.size(0), -1)
    x = self.relu(self.fc1(x))
    x = self.fc2(x)
    return x

model = SimpleCNN().to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

num_epochs = 5
train_losses = []

print("\n--- Начало обучения ---")
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        # Прямой проход
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Обратный проход и оптимизация
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)
    train_losses.append(epoch_loss)
    print(f"Эпоха [{epoch + 1}/{num_epochs}], Ошибка (Loss): {epoch_loss:.4f}")

model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f"\nТочность модели на тестовой выборке (Accuracy): {accuracy:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(range(1, num_epochs + 1), train_losses, marker='o', color='b', label='Train Loss')
plt.title('График изменения ошибки обучения (Loss)')
plt.xlabel('Эпоха')
plt.ylabel('Функция потерь (Cross Entropy)')
plt.grid(True)
plt.legend()
plt.show()

def predict_random_image(dataset, model):
    model.eval()
    idx = np.random.randint(0, len(dataset))
    image, label = dataset[idx]

    input_tensor = image.unsqueeze(0).to(device)

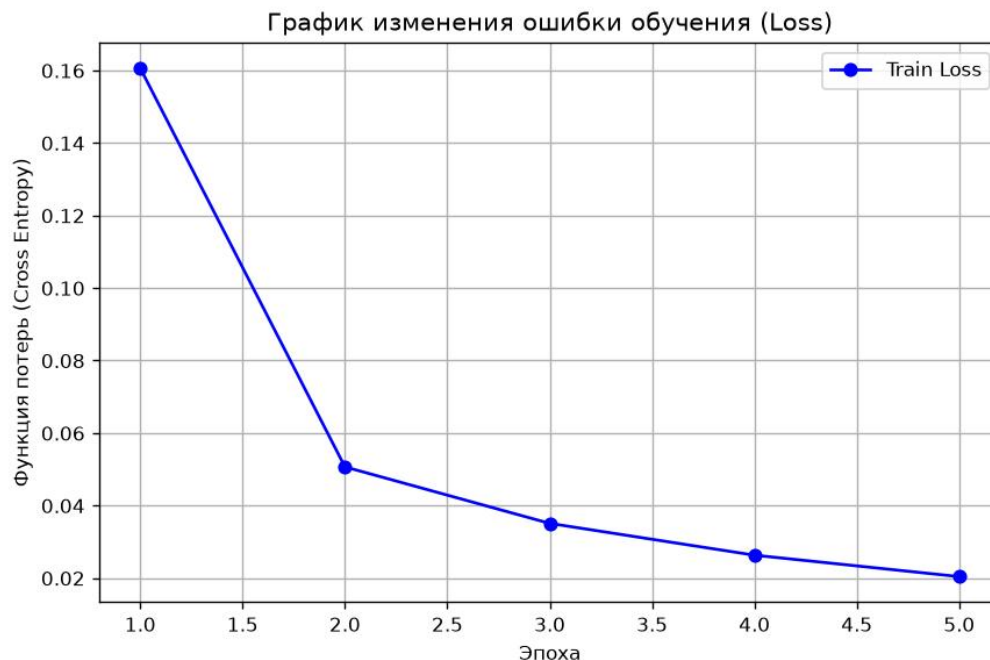
    with torch.no_grad():
        output = model(input_tensor)
        _, predicted = torch.max(output, 1)

```

```
img_to_show = image.squeeze().cpu().numpy()
plt.figure(figsize=(4, 4))
plt.imshow(img_to_show, cmap='gray')
plt.title(f"Истинная метка: {label} | Предсказание СНС: {predicted.item()}")
plt.axis('off')
plt.show()
```

```
predict_random_image(test_dataset, model)
```

Результат программы:



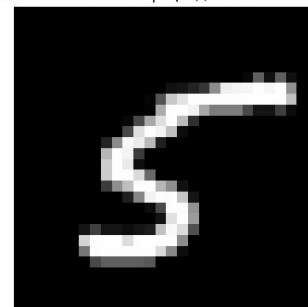
```
Используем устройство: cpu

--- Начало обучения ---
Эпоха [1/5], Ошибка (Loss): 0.1606
Эпоха [2/5], Ошибка (Loss): 0.0506
Эпоха [3/5], Ошибка (Loss): 0.0350
Эпоха [4/5], Ошибка (Loss): 0.0263
Эпоха [5/5], Ошибка (Loss): 0.0204

Точность модели на тестовой выборке (Accuracy): 98.99%

Process finished with exit code 0
```

Истинная метка: 5 | Предсказание СНС: 5



Вывод: в ходе выполнения лабораторной работы была спроектирована и обучена сверточная нейронная сеть для задачи классификации рукописных цифр из набора данных MNIST. Использование базовой архитектуры, состоящей из двух сверточных блоков и полносвязного классификатора, в сочетании с оптимизатором Adam и функцией потерь CrossEntropyLoss, позволило добиться высокой скорости сходимости сети. За 5 эпох обучения значение функции потерь на обучающей выборке уверенно снизилось с 0.1606 до 0.0204, а точность классификации на контрольной тестовой выборке составила 98.99%. При сравнении с передовыми мировыми результатами (State-of-the-Art) на датасете MNIST, составляющими 99.84%–99.87%, разработанная модель показала отличный результат, уступив сложным SOTA-архитектурам менее 0.9%. Разница обусловлена тем, что в SOTA-решениях применяются аугментация данных, ансамблирование моделей и более глубокие сети с регуляризацией. Таким образом, сконструированная лаконичная модель продемонстрировала оптимальный баланс между вычислительной сложностью и точностью, подтвердив высокую эффективность применения сверточных нейронных сетей для задач компьютерного зрения.